

遮住光，電腦怎麼知道？

Week 3：用電表看到電壓，再讓 ESP32 讀到變化。

上週按下按鈕，電腦出現「按下」或計數增加。這週不按按鈕，改用一張不透光的紙，看看電腦能不能分辨室內光與遮光。



我們依序做	要看到的結果
用電表量 3V3 和 GPIO5	電源電壓，以及程式切換的高、低電壓
接光敏模組，再用紙遮住	電表讀值及電腦數字是否改變
用自己的數據設定規則	畫面出現「室內光 / 遮光 / 未判定」
交換兩顆固定電阻	解釋光敏模組為何能改變電壓

今天不用馬達、舵機、Wi-Fi、OLED 或蜂鳴器。先完成「感測 → 程式判斷 → 紀錄」；它是之後軟體控制作品的基礎。

一起做的方式：跟著每頁的接線、動作與預期結果同步操作。做完一小段，再讀「做完再講」。尚未完成的數值留白，不抄示例當實測。

收起按鈕，換一個問題

GPIO4、GPIO5 這週要換用途，不能沿用計數器接線。

1. 拔掉 ESP32 的 USB，移除電池及其他外部供電，確認 PWR 燈熄滅。
2. 電表轉 OFF，移開表筆。拍下舊作品接法後，拆除開發板所有外接線。
3. 拿空的 400 孔麵包板。板子放在乾燥、不導電桌面，不直接插進麵包板。
4. 按鈕先收起來；本週不需要按住任何按鈕來量電壓。

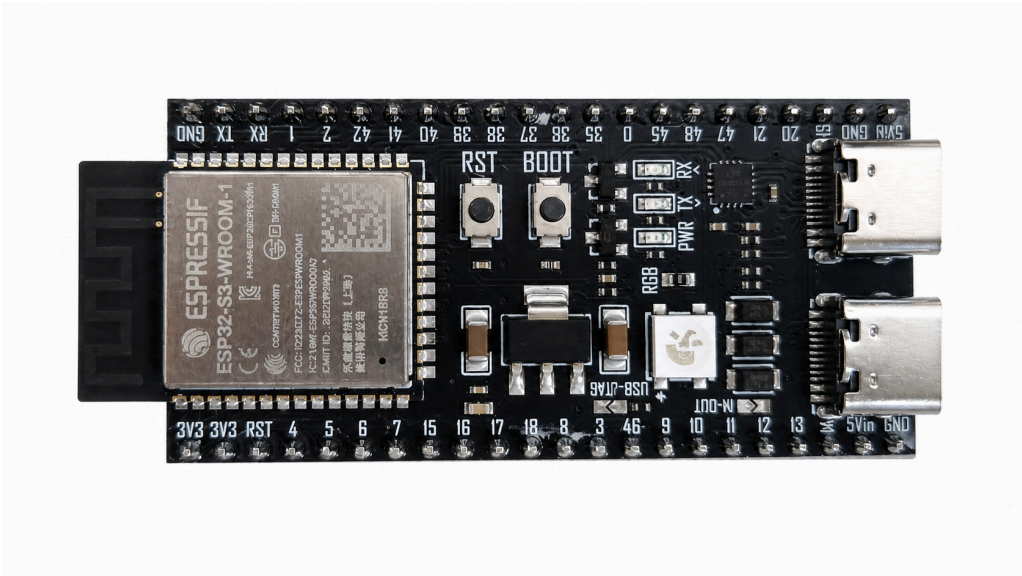
現在拿出來	數量
圖示 ESP32-S3 開發板、USB 資料線、筆電	各 1
萬用電表、400 孔麵包板	各 1
公對母 / 公對公 / 母對母杜邦線	各數條
KY-018 光敏模組、不透光的紙	各 1
電阻包中的 1 k Ω 、10 k Ω	各 1 顆

以後每次改線：先移開表筆 → 拔 USB 與其他電源 → 確認 PWR 熄滅 → 電表 OFF → 才移線。關閉 Serial Monitor 不等於斷電。

本週三個用途：3V3 給小型感測模組供電；GPIO5 暫時輸出高 / 低電壓；GPIO4 稍後讀光敏訊號。這些用途不是同時接上舊按鈕。

只找四個腳位

天線在左，USB 在右。看印字，不是數第幾根針。



課堂實物正面：YD-ESP32-S3 Type-A V1.5 · N16R8。

找哪個印字	這張照片的位置	用途
GND	上排最左端，靠天線	共同參考點
3V3	下排最左端	約 3.3 V 電源
5	下排，在 4 與 6 之間	稍後輸出電壓
4	下排，在 RST 與 5 之間	稍後讀光敏訊號

不要接 5Vin。 這片板的 IN-OUT 焊盤未橋接時，5Vin 不是 USB 的 5 V 輸出。本週不焊接、不橋接它。

本稿接法對應圖示板型。不同板型、看不清印字或課程尚未確認 GPIO 用法時，保持斷電，一起核對。照片不能證明所有同名商品相同。

下一步：先不用 GPIO，從 3V3 與 GND 量一次電壓。

黑筆、紅筆、旋鈕各放哪裡？

這次量電壓，不是量電流，也不是聽通斷聲。

V 是電壓單位「伏特」。20 V 檔是量測範圍，不是電表會輸出 20 V。



課堂 A830L 實物照片。照片旋鈕在 OFF；量測前仍須自己選檔。

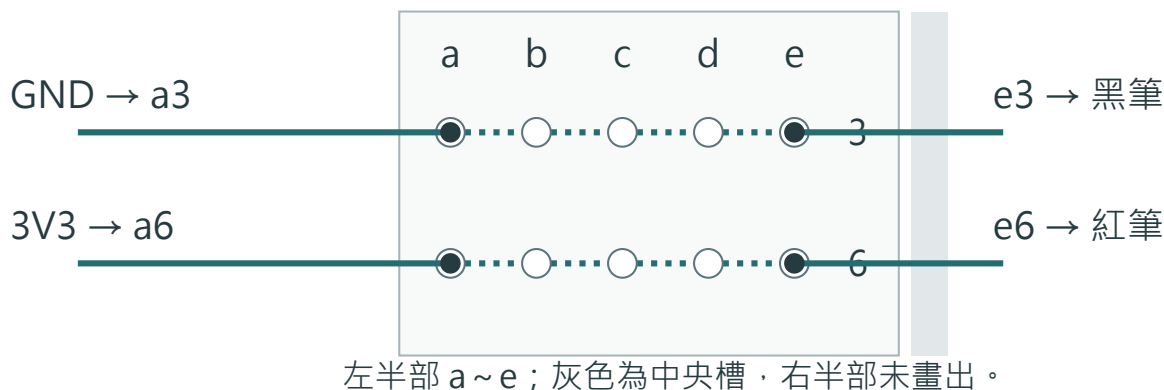
位置	這次設定
黑表筆插孔	COM；此表下方中間
紅表筆插孔	VΩmA；此表下方右邊
旋鈕	直流電壓 20 V：找 V 與實線 / 虛線符號，不是 V~
10A 插孔	保持空著

COM 是電表的共用參考端，並不會自己連到板子的 GND。等一下要由黑表筆碰到 GND 的延長線，兩者才連起來。

本週完全不使用 A / mA 電流檔，也不量插座或市電。表型不同時看自己的符號與說明，不照照片角度猜。先設好，再將表筆放旁邊。

兩條線，把電源位置引出來

麵包板只讓表筆比較好碰，不會自己製造電源。



先後	起點 → 終點	接頭
1	ESP32 GND → a3	母頭套排針，公頭插孔
2	ESP32 3V3 → a6	母頭套排針，公頭插孔
3	e3 → 一條公對公線的自由端	黑筆稍後碰這個自由端
4	e6 → 另一條公對公線的自由端	紅筆稍後碰這個自由端

同列 a~e 相通。a3 與 e3 是同一位置的延伸；a6 與 e6 是另一組。第 3 列和第 6 列不可用線直接相連。兩個自由金屬端要分開固定。

一起核對表格及列號，確認沒有其他線或元件跨接第 3、6 列。若做通斷複查，只能在拔 USB 時做；供電與 GND 持續蜂鳴時保持斷電排查。**不蜂鳴也不是完整安全保證。**

表筆不要硬塞進麵包板。不要讓延長線金屬端互碰，也不要一邊通電一邊整理線。

黑筆固定，紅筆只碰一個端點

先猜：3V3 相對 GND，會接近 0 V 還是 3.3 V？

1. 電表確認為直流 20 V。插上板背標示 COM 接頭的 USB。
2. 黑筆碰 e3 延長線的自由金屬端。
3. 紅筆碰 e6 延長線的自由金屬端，不碰旁邊排針。
4. 接觸穩定後記錄完整畫面與單位，再移開紅筆。



顯示例：約 +3.3 V；兩端不可用普通線直接互接。

黑筆	紅筆	檔位	預期	我的結果
e3 · GND	e6 · 3V3	直流 20 V	約 +3.3 V	___ V

課堂舊紀錄曾回報約 3.2 V；那是另一輪操作，不是你應照填的答案，也不是本次實測。

如果是負數：先移開表筆，核對紅黑測點，不交換板子的供電線。**如果接近 0 V：**移開表筆、斷電，核對 USB、PWR、3V3 印字與列號。

下一頁不改線，先解釋剛才這個數字。

電壓是比較兩個位置

不是「紅筆裡面有 3.3 V」，也不是「流過了 3.3 A」。

電表在直流電壓檔顯示：**紅筆位置的電位** – **黑筆位置的電位**。本週把 GND 當共同的 0 V 參考。

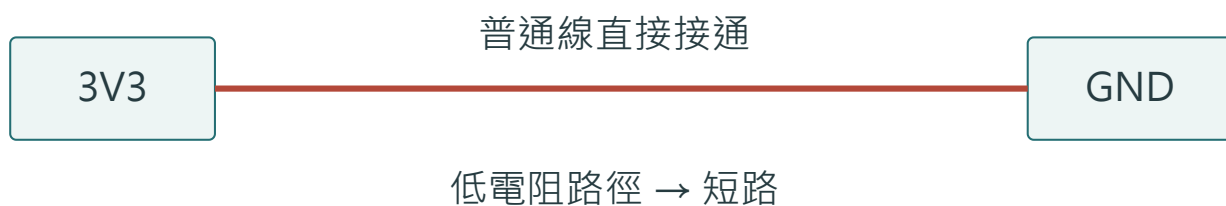
紙上情境，不必重新接線	計算	顯示
紅筆 3V3、黑筆 GND	$3.3 - 0$	+3.3 V
紅筆 GND、黑筆 3V3	$0 - 3.3$	-3.3 V
兩筆碰同一個 3V3 節點	$3.3 - 3.3$	0 V

第三列不表示板子沒電，只表示兩個測點的電位相同。GND 也不是讓電流憑空消失的地方，它接回電源回路。

正常量測：



不要實作：



為什麼電表跨接 3V3 和 GND 不等於短路？電壓檔內部有很大的輸入電阻，只取很小的電流。普通杜邦線的電阻很低，不能拿來代替電壓表。

上圖只是比較電路，不實作直接短接。若把電表切到電流檔，內部路徑不同，就不能照剛才的方式跨接電源。

換一支會切換電壓的程式

上週 GPIO5 是減一按鈕；現在先讓它成為輸出。

1. 移開表筆、拔 USB、電表 OFF。從開發板拔下外接線；麵包板的延長線可以留著。
2. 在 Arduino IDE 選 **File → Open**，開啟隨課程提供的 [GPIO 電壓程式](#)。完整程式也在本冊後方。
3. 找到 `PIN_TEST_OUTPUT = -1`。本稿示範接點是 **GPIO5**；只有課前已確認使用圖示板型與這個接點時，改為 5。未確認保留 -1，不猜其他數字。
4. 板子只插 COM 接頭 USB，依下表選設定，按左上方 **右箭頭 Upload**。

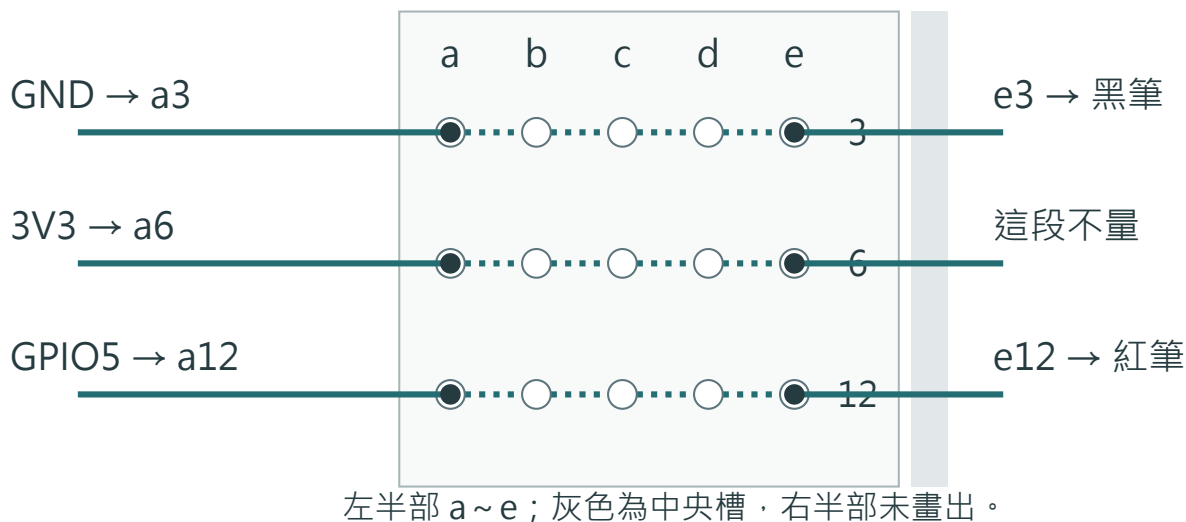
IDE 位置	本課圖示板設定
Tools → Board	ESP32S3 Dev Module
Tools → Port	插入此板後新增的 COM，號碼不固定
Flash Size / PSRAM	16MB / OPI PSRAM
USB CDC On Boot / USB Mode	Disabled / Hardware CDC and JTAG
Upload Mode / Upload Speed	UART0 / Hardware CDC / 115200
Flash Mode / Partition Scheme	QIO / 16M Flash (3MB APP/9.9MB FATFS)

按 **Tools → Serial Monitor**，速度選 **115200**。預期看到 `startup=LOW`，之後約每 10 秒出現 LOW 或 HIGH。出現 `gpio_profile_missing` 是腳位仍未填，不是電路壞了。

本稿沿用 Week 2 的 IDE 安裝與 Espressif 套件。尚未安裝者先完成 Week 2 安裝段；這一頁是重新選板、選連接埠與換程式的完整提醒。上傳後先拔 USB，再接下一頁。

加一條 GPIO5 線，量它的電壓

沒有按鈕、LED 或其他負載；GPIO5 只接電表量測。



起點 → 終點	現在的作用
GND → a3 ; e3 → 黑筆延長端	恢復共同參考點
3V3 → a6 ; e6 延長端分開收妥	恢復電源測點，這段不量它
GPIO5 → a12	新增程式控制的測點
e12 → 新的公對公延長線	紅筆稍後碰這個端點

每條外接線照表恢復，特別確認 GPIO5 不是 5Vin。第 12 列不能直接接到第 3 列或第 6 列。

這就是舊教材的 TPO：只是「方便量 GPIO 輸出的地方」，不是另一個電子零件，也不是資料到電腦必經的位置。本稿直接稱它 **GPIO5 / 第 12 列**。

先確認舊按鈕已拆除。GPIO5 若輸出 HIGH 時仍被按鈕直接接 GND，會造成過大電流的風險。不要沿用計數器接線。

LOW、HIGH，電表真的不同嗎？

先猜：哪個階段接近 0 V？哪個接近 3.3 V？

1. 確認接線後插 COM USB，開 Serial Monitor，選 115200。
2. 電表選直流 20 V。黑筆碰 e3 的延長端；紅筆碰 e12 的延長端。
3. 看 phase=LOW 或 phase=HIGH，等讀值穩定再記錄。不要把切換瞬間的跳動值當穩態。

次數	phase	預期電壓	實測電壓
1	LOW	約 0 V	___ V
1	HIGH	約 3.3 V	___ V
2	LOW	約 0 V	___ V
2	HIGH	約 3.3 V	___ V
3	LOW	約 0 V	___ V
3	HIGH	約 3.3 V	___ V

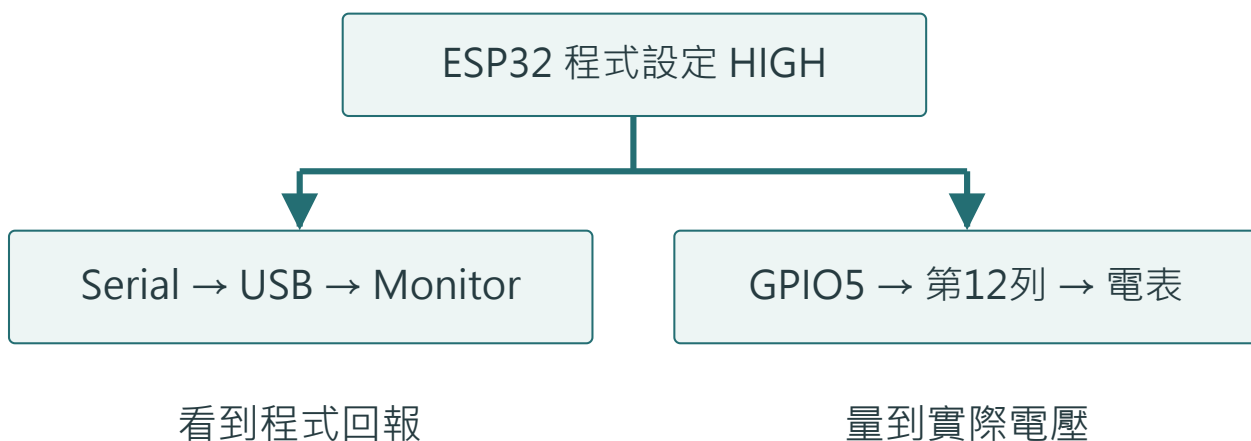
再看一次啟動：移開兩支表筆，按一下板上的 RST，保存重新出現的 `startup=LOW` 文字。這裡只看啟動紀錄，不重做整張電壓表。

``expected_voltage=near_3.3V`` 是程式寫出的預期，不是 ESP32 已幫你量過。電表穩態讀值也不能證明 Reset 瞬間完全沒有短脈衝。

完成後拔 USB、電表 OFF。下一頁不接新線，先看兩份證據的差別。

一個命令，兩種證據

Serial 說「我要求 HIGH」；電表說「我量到幾伏特」。



程式中的動作	對應剛才哪件事
<code>pinMode(..., OUTPUT)</code>	指定 GPIO 用來輸出，不再讀按鈕
<code>digitalWrite(..., HIGH/LOW)</code>	要求輸出高 / 低準位
<code>millis()</code> 的時間差達 10000	約等 10 秒才切換一次
<code>Serial.printf(...)</code>	把命令與次數送到電腦

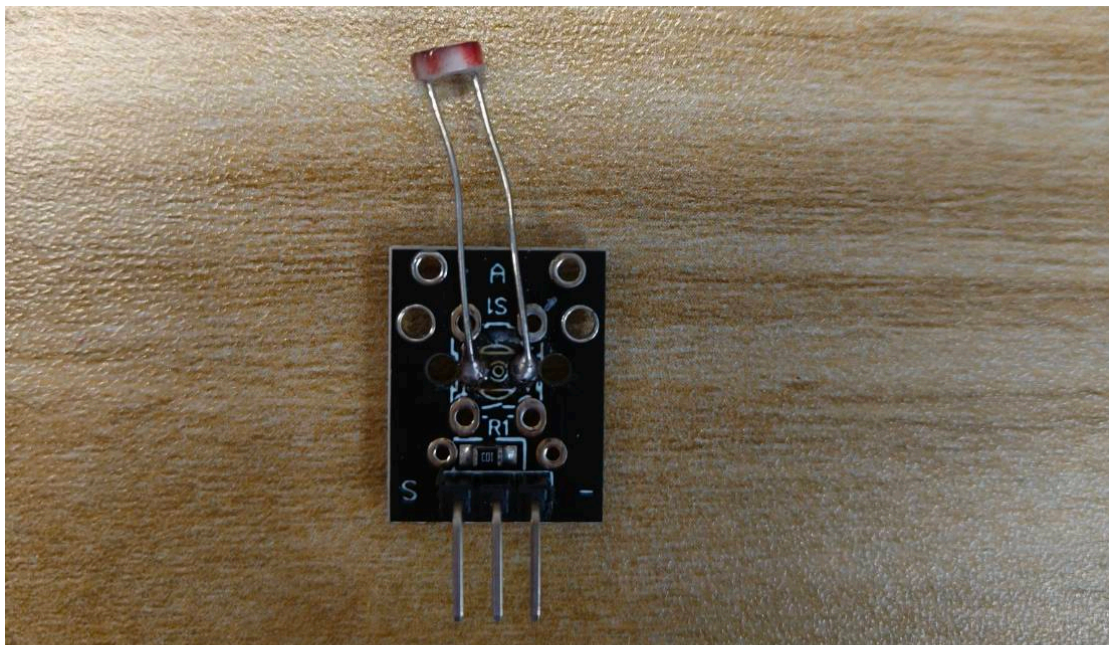
3V3 和 GPIO5 不相同。3V3 是供電腳；GPIO5 是訊號腳，電壓由程式控制。讀到約 3.3 V，不代表 GPIO 可以當一般 3.3 V 電源帶動馬達或模組。

自己回答：如果 Monitor 顯示 HIGH，電表卻是 0 V，能說程式完全正確、接線也正確嗎？先斷電查腳位、列號和上傳版本，不能只相信文字。

下一段要讀光敏訊號。保持斷電，拆除 GPIO5 → a12 與 e12 延長線；3V3、GND 兩列稍後繼續使用。

找到會感光的元件和三支腳

模組先放桌上，不插麵包板，不接 USB。



實物照片局部放大，原圖未修改：圓形感光元件朝上，三支排針朝下。

照片中的排針	已有實物紀錄的用途
左邊，獨立 S 標示旁	訊號，稍後量它的電壓
中間，沒有獨立供電標示	本課已核對模組的供電端
右邊，獨立 — 標示旁	接 GND

先一起比對自己的絲印和照片。板上 A、S1、R1 是元件區標示，不能直接當排針名稱。下一頁在斷電下量三組路徑，確認和既有接法相符。

照片朝向改了，左右也會改。不同模組不能只照左右猜供電。本稿只適用已確認「中間到 S 是固定電阻、S 到 — 是光敏電阻」的接法；結果不符就不供電。

不接電，也能先看阻值變化

這裡改用 Ω 檔；ESP32、USB、電池都不能接在模組上。

Ω 是電阻單位「歐姆」；k 表示一千，所以 $1\text{ k}\Omega = 1000\text{ }\Omega$ 。200k Ω 是量程，不是模組一定有 200 k Ω 。

1. 黑筆仍插 COM，紅筆仍插 V Ω mA。先選 **200k Ω** ，不是直流 20 V。
2. 黑、紅筆各碰指定排針，手只握絕緣部分。固定接觸後，用紙遮住感光面，不碰動排針。
3. 同一組測點依序比較「室內光 → 遮光 → 移開紙」。換測點時也保持模組不接電源。

測哪兩腳	室內光	遮光	移開紙
中間 ↔ S	___ k Ω	___ k Ω	___ k Ω
S ↔ -	___ k Ω	___ k Ω	___ k Ω
中間 ↔ -	___ k Ω	___ k Ω	___ k Ω

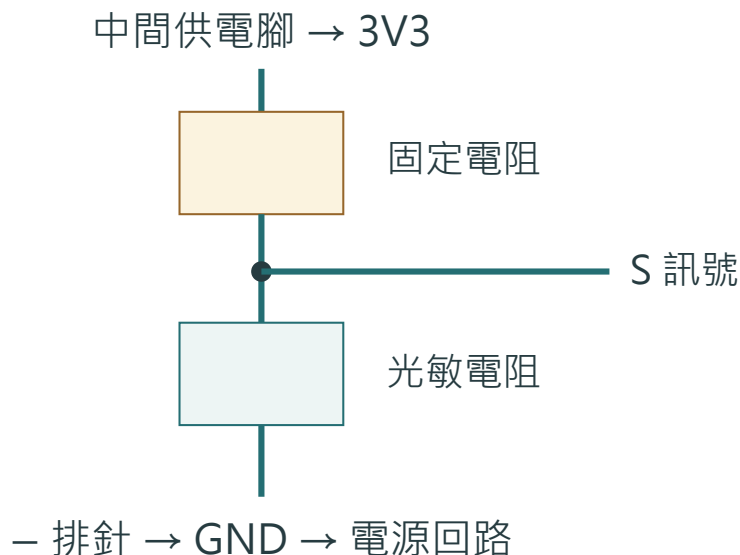
記下**檔位、完整小數點與單位**。200k 檔例如顯示 10.0，通常是 10.0 k Ω ，仍以自己的表型說明為準。若超出量程顯示 1 / oL，不要寫成 1 Ω ；提高量程或保存「超量程」。

先看變化，不求精密驗證：本課既有接法預期中間 ↔ S 比較固定；S ↔ - 隨光線改變。沒有變化先查遮光與接觸，不交換供電腳來試。

量完電表 OFF，表筆放旁邊。下一頁把三組結果放回電路。

光敏模組裡有兩個電阻

剛才量的三組腳，是兩段電阻及它們的總路徑。



上面的固定電阻：接在中間供電腳與 S 之間。**下面的光敏電阻：**接在 S 與 — 之間。S 是兩者中間的位置，不是新的電源。

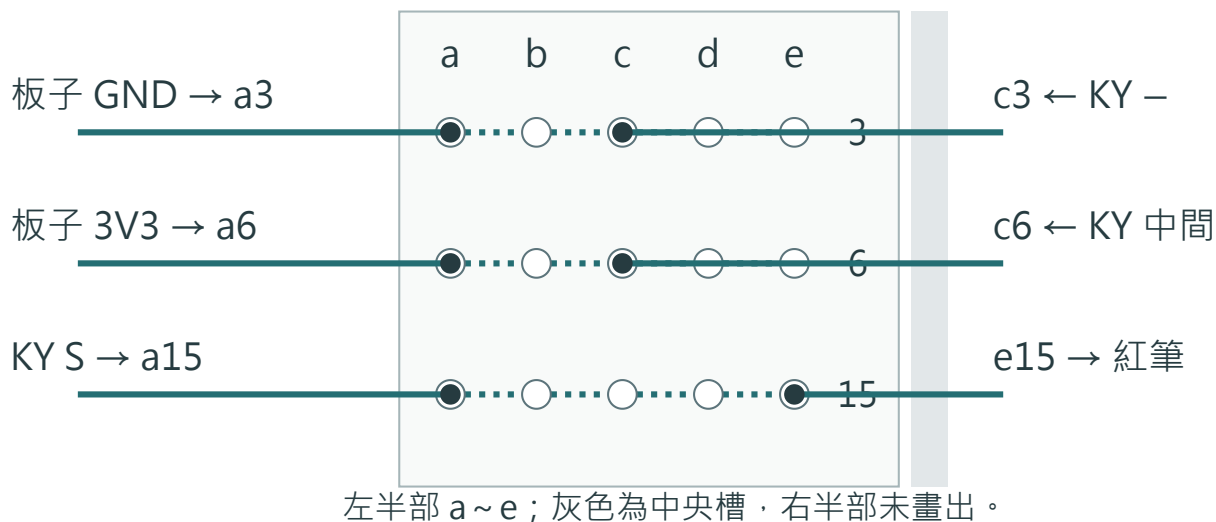
剛才量的路徑	這張圖的對應
中間 ↔ S	上面一段
S ↔ —	下面一段
中間 ↔ —	兩段串聯的總路徑

同樣光照與接觸下，總阻值應大致接近兩段相加。三次量測若光照變了、手碰到金屬或量程不同，不能把差異當成精密驗證通過。

先保留一個問題：兩段之間的 S，供電後會是多少 V？不用先背公式，下一段直接量。公式等固定電阻實作後再算。

先接供電，再把 S 引出來

這一步只給模組供電。S 還不接 GPIO4。



起點 → 終點	說明
ESP32 GND → a3 ; 3V3 → a6	保留前段兩條電源線
KY 中間供電腳 → c6	用母頭套模組腳，公頭插孔
KY - → c3	接到同一個 GND
KY S → a15	只是引出訊號，不接 GPIO
e15 → 新的公對公延長線	紅表筆的訊號測點
e3 → 黑筆延長線	沿用 GND 測點

GPIO5 與第 12 列的線已拆除。e6 電源延長線這段不用，斷電時拆下，避免紅筆量錯。模組放不導電桌面，不把三支腳插在麵包板同一個五孔組。

一起核對中間腳、S、- 及列號。S 不直接短接 GND；GPIO4 暫時空著。只用 3V3，不用 5Vin、5V 或其他電池供電。

遮住之後，電壓升還是降？

先猜一次，再用自己的讀值回答。

1. 電表選 **直流 20 V**，不能留在上一段的 Ω 檔。
2. 接 **COM USB**。黑筆碰 e3 延長端；紅筆碰 **e15** 延長端，不是 e6。
3. 不移動模組，記錄室內光；用不透光紙遮住感光面，再記錄；移開紙，最後再記一次。

先後	條件	我的 S 對 GND 電壓
1	固定位置、室內光	___ V
2	原位置、用紙遮住	___ V
3	原位置、移開紙	___ V

我的觀察：遮光後電壓 _____；移開紙後。板子、檯燈、紙的位置與表筆接觸是否都固定？。

本課舊紀錄曾觀察遮光升高，較新的組裝卻回報遮光數字降低。兩輪不能合併成一組答案。本次只依自己的接法與同輪資料判斷，不預先規定「暗一定比較大」。

讀值超出 0~3.3 V、負值或異常發熱：先移開表筆、斷電，查供電與測點。正常觀察完成後也先移開表筆、拔 USB、電表 OFF。

下一步：目前是人看電表；接下來讓 ESP32 讀同一個 S。

把 S 的電壓變成電腦上的數字

先換程式，再把訊號接入 GPIO4。不要沿用 GPIO 輸出程式。

1. 保持斷電，從開發板拔下外接線。麵包板與模組間的線可保留，沒有電源供應。
2. IDE 選 **File** → **Open**，開啟 [光敏原始讀值程式](#)。
3. 找 `PIN_LIGHT = -1`。課前已確認本稿的 GPIO4 接法時改成 4，否則保留 -1。不要沿用上一支程式的 5。
4. 將 `DEVICE_ID = "CHANGE_ME"` 改成自己的組別代碼，例如 "G01"。不要放姓名或學號。
5. 板子只接 COM USB，依第 8 頁重新核對板型與 Port，按 Upload。

Serial Monitor 選 115200。看到 `status=ready` 及 `interval_ms=500`，表示程式已開始準備取樣，不代表線已接對。

```
device=G01 ... adc_bits=12 interval_ms=500
```

這是欄位示意，不是本次實測。尚未接模組的輸入可能亂跳，**現在的 raw 不拿來當光線資料**。

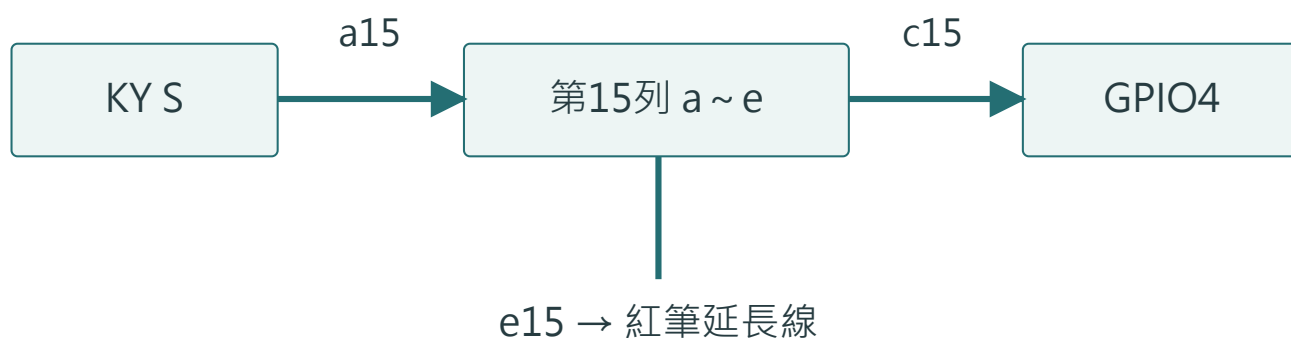
ADC 是晶片裡把電壓轉成數字的功能。現在先把讀值做出來；0 ~ 4095 與電壓的差別，等看到數字後再解釋。

完成後拔 USB。下一頁恢復電源線，再加唯一一條新訊號線。

多一條線：c15 接到 GPIO4

第 15 列是同一個 S 訊號，電表和 ESP32 可以看同一位置。

新增訊號線；模組供電接法保持不變



黑筆 → e3 / GND；模組 — → c3 / GND。

核對這些起點 → 終點	現在要做
ESP32 GND → a3；3V3 → a6	接回剛才拔下的電源線
KY — → c3；中間 → c6；S → a15	維持上一段接法
c15 → ESP32 GPIO4	新增公對母線；不是 GPIO5
e15、e3 的兩條延長線	分開固定，供稍後量 S / GND

a15、c15、e15 是同一五孔組的三個入口。不是三個不同訊號，也不需要再把 S 直接接一條線到 GND。

GPIO5 保持空著；4 不是 3V3。所有訊號只能在已確認的安全電壓內。ESP32-S3 GPIO 不接受 5 V 訊號；本週模組以 3V3 供電。

全班對照接點後插 COM USB，開 Monitor，選 115200。先固定模組、遮住、移開一次，看數字是否改變，再開始保存資料。

室內光十筆，遮光十筆

不是只挑最好看的兩個數字，也不是把不同位置的資料拼起來。

1. 模組、桌燈、電源、接線位置不變。先預測遮光後 raw 比室內光大還是小：_____。
2. 室內光穩定後，從 Monitor 保存連續 **10 行**，標記「室內光」。保留序號、時間及 raw。
3. 用同一張紙在原位置遮住感光面。穩定後另存連續 **10 行**，標記「遮光」。
4. 每種條件也量一次 S 對 GND：直流 20 V，紅 e15、黑 e3。先量再存 log，不宣稱兩者精確同步。不要帶電移線。

```
device=G01 sample=21 uptime_ms=10500 light_raw=310
device=G01 sample=22 uptime_ms=11000 light_raw=314
```

上面是**示意輸出**。只讀最後的 light_raw 作光線原始值；sample 是序號，uptime_ms 是開機後毫秒。

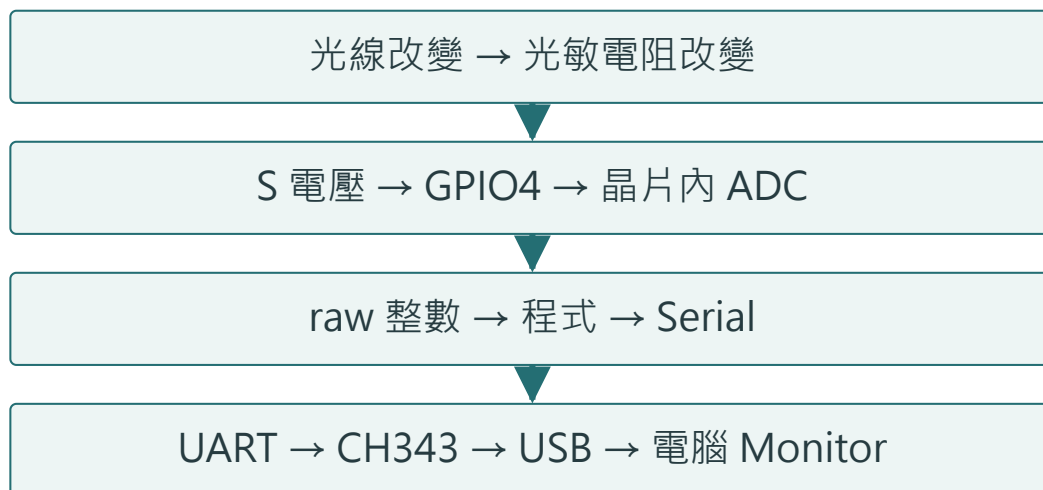
條件	10 筆最小值	10 筆最大值	電表
室內光	_____	_____	_____ V
遮光	_____	_____	_____ V

保存全部 **20 行**，表格只是摘要。500 ms 是 0.5 秒；10 筆有 9 段間隔，首尾約 4.5 秒。若中途重開機或移位置，另開一輪，不混算。

已有同輪完整資料可沿用；不足的地方寫未完成，不抄教材湊筆數。

raw 不是伏特，也不是亮度單位

電表顯示 V；程式目前顯示的是 ADC 原始整數。



圖中的 UART 是 ESP32 傳送這些文字的通訊介面；CH343 是圖示板 COM 接頭使用的 USB 轉接晶片。本頁是資訊路徑，不是新增接線指令。

12-bit 在這支程式中代表 **4096 個編號**，從 0 到 4095。raw=700 不是 700 V、0.700 V，也不是 700 lux。

程式設定 ADC_11db；Espressif 文件列 ESP32-S3 此設定的可量測範圍約 0~3.1 V。靠近端點可能飽和，不能假設 4095 正好等於 3.3 V，也不能保證每一格都同樣精準。

現在可以說	不能直接說
此輪遮光後，我的 raw 上升 / 下降	所有 KY-018 都同方向
電表與 raw 在此輪大致同向改變	我已精確校正 ADC 電壓
接法相同時，數字可用來比較	數字就是照度 lux

一起解釋：用自己的 20 行資料回答「raw 變大代表比較亮嗎？」先說哪個條件，再指出兩組範圍，不能只回答是或不是。

模組 — 與板子 GND 是共同參考；訊號送到 GPIO4。ADC 內部完成比較，不是叫你在板外用線把 GPIO4 和 GND 短接。

先畫兩段範圍，再選分界

用剛才資料，不重新收一樣的二十筆。

教學假資料；raw 無單位，不是 V



示意位置非等比例。兩段之間的值沒有基準資料。

上圖是**教學假資料**：室內光 300 ~ 320，遮光 900 ~ 920。兩段沒有重疊，中間可以選一個分界：

分界 = (室內最大值 320 + 遮光最小值 900) ÷ 2 = 610。在這個方向下，raw ≤ 610 判室內光，raw > 610 判遮光。

你的資料狀態	接下來怎麼做
遮光整段比室內光大	用室內最大值與遮光最小值求中點
遮光整段比室內光小	用遮光最大值與室內最小值求中點；方向反過來
重疊，或兩段碰到同一數字	目前無法用這個分界分清，先保存結果，不硬設

填自己的四個值：室內最小 ____、室內最大 ____、遮光最小 ____、遮光最大 ____。我的分界 ____；遮光是較大 / 較小 / 還無法分開。

最小 / 最大值是這一輪的範圍，不是未來永遠的界線。不要刪掉不合預期的數字，只為了讓範圍分開。

讓程式印出「室內光 / 遮光」

這次 GPIO4 仍是輸入，模組接法不變。

先移開表筆、電表 OFF。關閉上一個 Monitor，IDE 開啟 [光線分類程式](#)，只填下面這些設定：

程式名稱	填什麼
PIN_LIGHT	與上一支原始讀值程式相同、已確認的腳位
DEVICE_ID	同一組別代碼
INDOOR_MIN、INDOOR_MAX	自己室內光十筆的最小、最大值
SHADE_MIN、SHADE_MAX	自己遮光十筆的最小、最大值

分界與方向由程式算，不必另把 610 抄進程式。確認第 8 頁設定後 Upload，Monitor 選 115200。

```
status=ready ... threshold=610 shade_higher=true
... light_raw=310 ... class=INDOOR label=室內光
```

上面沿用前頁假資料示範格式，**你的分界應來自自己的資料**。若出現 `calibration_missing_or_overlap`，表示有缺值、不合範圍或重疊；回前頁查四個設定，不拆線試錯。

既有分類程式保守排除 `raw=0、4095` 的基準。這是課堂規則，不代表端點讀值一定是斷線。尚未取得可分開的基準時，保留原始資料與原因就好，不偽造通過。

規則設好了，不代表一定判對

不用剛才建立規則的二十筆，改看接下來的新資料。

1. 先在相同位置回到室內光，保存 5 筆新紀錄。
2. 再以相同方式遮光，保存 5 筆新紀錄。
3. 每筆比較「你實際做的動作」與 label，也保留 quality、reason。

真實操作條件	新資料筆數	標籤相符	不相符 / 未判定
室內光	5	——	——
遮光	5	——	——

例：若 10 筆有 8 筆標籤相符，可以說「這次、這個位置的 10 筆有 8 筆相符」。不能說整個系統準確率永久 80%，也不能把短時間連續取樣視為十種環境。

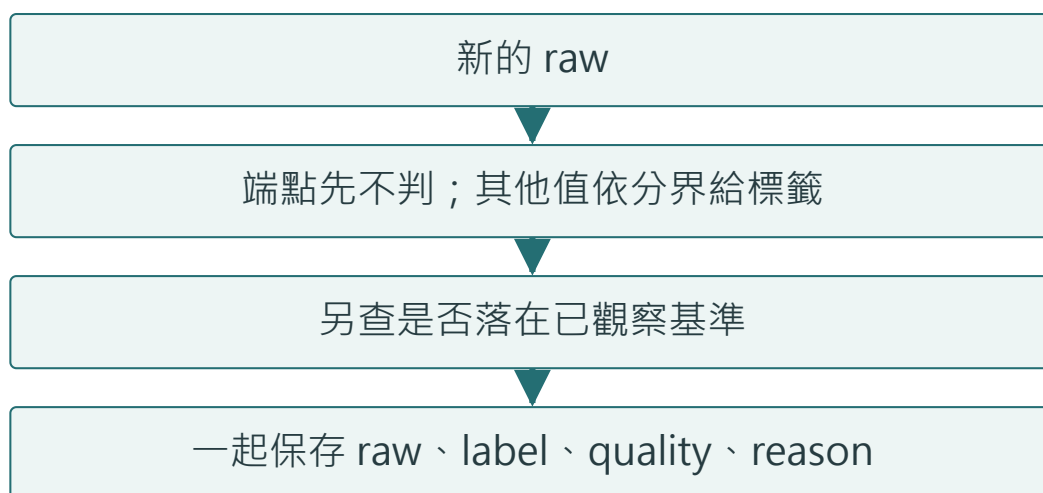
若全部都判同一類：先對照原始值與四個範圍，確認條件、上傳版本、模組位置與組別資料。不要先交換電源線。

保留另一個問題：同一個 raw 落在兩組範圍之間時，程式雖然印了標籤，我們真的看過這個數字所代表的環境嗎？下一頁拆開看。

看 raw，也看為什麼這樣判

沿用假資料：室內 300 ~ 320、遮光 900 ~ 920、分界 610。

新 raw	程式標籤	quality / reason 的意思
310	室內光	relative_only：在已觀察的基準內，仍只是本地相對判斷
700	遮光	suspect / between_baselines：落在兩段之間，沒有基準支持
1000	遮光	suspect / outside_observed_span：超出曾觀察的總範圍
0 或 4095	未判定	suspect / adc_endpoint：端點先不分類，不等於證明斷線



這些英文是程式輸出的欄位值：raw 是原始數字，label 是規則給的文字，reason 是這筆數字和基準的關係。它們不能互相替代。

討論：假設剛才遮光較大，現在換位置卻較小，先查哪兩件事？先保存兩輪，不合併；再查接法 / 模組是否相同，以及光線、遮法和接觸是否改變。找到差異前不能套用舊規則。

下一段斷電後拆下光敏模組。用兩顆固定電阻，把「為什麼電壓會變」拆成看得見的電路。

拿出 1 k Ω 和 10 k Ω

先量實物，再接電路。不是只看包裝就假設一定正確。

移開表筆，拔 USB，電表 OFF。拆下 KY 的供電、GND、S 線，以及 GPIO4 線與第 15 列延長線。保留 GND → a3、3V3 → a6；板子仍斷電。



已購電阻包的實物與手寫標示。本段只取 1 k Ω 、10 k Ω 各一顆。

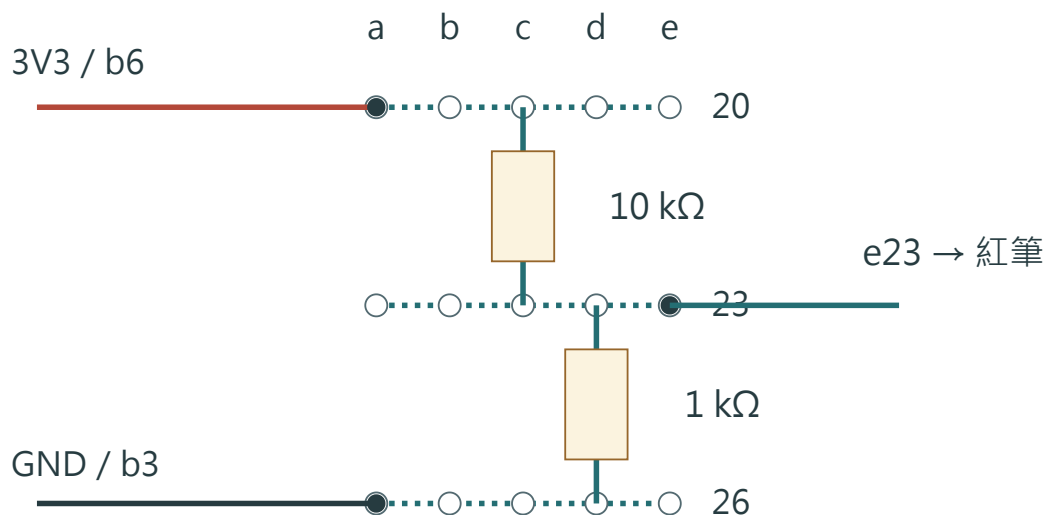
1. 每次拿一顆，與其他電路完全分離。
2. 黑筆 COM、紅筆 V Ω mA，選 20k Ω 。兩筆各碰電阻一端，不同時用手捏兩端金屬。
3. 記錄後電表 OFF。電阻不分正負，但接到電路的上下位置等一下會影響輸出。

包裝標示	完整畫面	換算後的阻值
1 k Ω	___	___ k Ω = ___ Ω
10 k Ω	___	___ k Ω = ___ Ω

1 k Ω = 1000 Ω 。理想 1 k Ω 在 20k 檔約顯示 1.00；理想 10 k Ω 約顯示 10.00。這是判讀示例，不是要求你的數值完全一樣。

先把 10 kΩ 放上面

上、下是電路的位置：靠 3V3 是上，靠 GND 是下。



黑筆仍碰 e3 延長端；電阻兩端跨不同列。

元件 / 線	兩端位置
供電延長線	b6 → a20
10 kΩ	c20 → c23
1 kΩ	d23 → d26
接地延長線	a26 → b3
紅筆用公對公延長線	e23 → 自由端
黑筆延長線	e3 → 自由端

第 23 列讓兩顆電阻接在一起。電阻的兩隻腳不能插進同一五孔組；上下端不能另加導線跨過電阻。輕彎腳配合孔距，不硬拉元件本體。

一起依表核對，確認 KY、GPIO4、GPIO5 都沒有接線。先猜：中間第 23 列會更接近 0 V，還是 3.3 V？寫下預測 ____ V，再翻頁量。

量到之後，再算一次

這裡只量中間電壓，不使用 ADC，也不量電流。

確認接法後插 USB。電表選直流 20 V，黑筆碰 e3 延長端，紅筆碰 e23 延長端。我的結果：
_____ V。保存接線照片後移開表筆。

為什麼預期約 0.3 V？ 電流先經上面 10 kΩ，再經下面 1 kΩ，最後回到 GND。兩顆串聯，流過它們的電流相同；比較大的電阻分到比較大的電壓差。

計算先定義：供電電壓 V_s ，上方電阻 $R_{上}$ ，下方電阻 $R_{下}$ ，中間相對 GND 的電壓 $V_{中}$ 。

$$\begin{aligned} V_{中} &= V_s \times R_{下} \div (R_{上} + R_{下}) \\ &= 3.3 \text{ V} \times 1 \text{ k}\Omega \div (10 \text{ k}\Omega + 1 \text{ k}\Omega) \\ &= 0.3 \text{ V} \end{aligned}$$

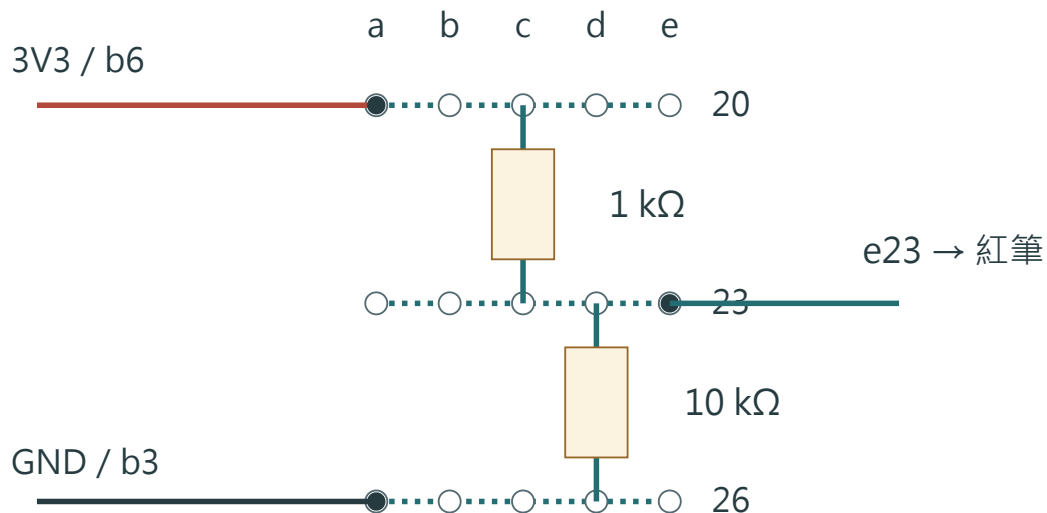
位置	理想電位	該電阻兩端差值
供電端 → 中間	3.3 V → 0.3 V	上方電阻：3.0 V
中間 → GND	0.3 V → 0 V	下方電阻：0.3 V

電表量的是中間到 GND，所以對應**下方電阻的電壓差**。若實際供電不是 3.3 V，用自己量的 V_s 與阻值重算，不把差異直接當成錯接。

下一頁要交換電阻。先拔 USB、確認 PWR 熄滅、電表 OFF。

交換位置，總電阻明明沒變？

原線不動，只交換兩顆電阻的上下位置。



黑筆仍碰 e3 延長端；電阻兩端跨不同列。

USB 保持拔除：把 $1\text{ k}\Omega$ 接 $c20 \rightarrow c23$ ； $10\text{ k}\Omega$ 接 $d23 \rightarrow d26$ 。其餘線沿用前頁。先預測中間電壓：_____ V。

核對後插 USB，直流 20 V，黑 e3、紅 e23。我的結果：_____ V。保存第二張接線照片，再移開表筆、拔 USB、電表 OFF。

$$\begin{aligned} V_{\text{中}} &= 3.3\text{ V} \times 10\text{ k}\Omega \div (1\text{ k}\Omega + 10\text{ k}\Omega) \\ &= 3.0\text{ V} \end{aligned}$$

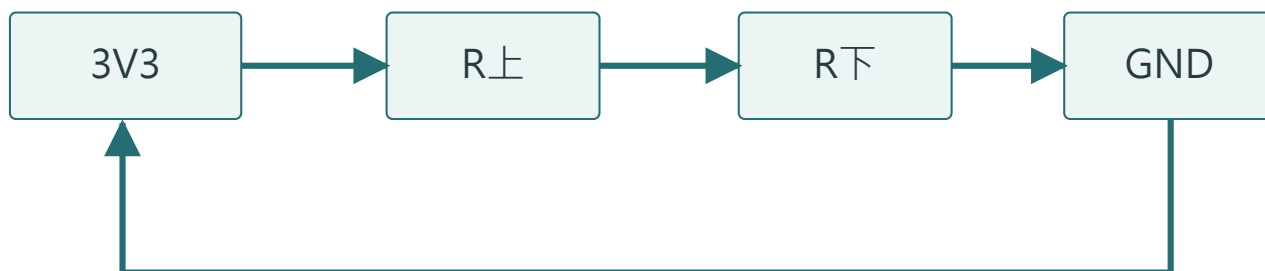
原來 $10\text{ k}\Omega$ 不是「固定扣掉幾伏特」。兩顆總阻值仍是 $11\text{ k}\Omega$ ；中間到 GND 現在跨的是 $10\text{ k}\Omega$ ，所以中間電壓不同。

回看 KY 模組：上方固定電阻不變，下方光敏電阻隨光改變，中間 S 的電壓就會跟著變。另一種內部接法可能方向相反，不能只憑商品名稱判方向。

同一條路，電流不會先被用掉

電阻限制整條串聯路徑的電流，不是把 3.3 A 變成小一點。

兩顆串聯電阻流過相同電流



經板上電源回路返回；畫的是傳統電流方向。

I 是電流，單位 A； V 是電壓差，單位 V； R 是電阻，單位 Ω 。歐姆定律在這個電阻電路中寫成 $I = V \div R$ ，不是 $V \times R$ 。

$$\begin{aligned} I &= 3.3 \text{ V} \div (10000 \text{ } \Omega + 1000 \text{ } \Omega) \\ &= 0.0003 \text{ A} = 0.3 \text{ mA} \end{aligned}$$

這是理想計算，不是電表實測電流。交換上下位置，總阻值不變，所以理想總電流仍相同。

紙上比較，不實際新增接線	判斷
3V3 → 兩顆串聯電阻 → GND	有控制電流的阻值，是本次正常電路
3V3 → 普通導線 → GND	電源被低阻路徑直接接通，這就是短路
路徑中拔掉一端	斷路，這條路不再形成完整回路

短路**可能**造成發熱、保護動作或損壞；不代表每次一定燒壞。沒有外加電阻也不等於 $R=1 \text{ } \Omega$ ，導線電阻可能遠小於 $1 \text{ } \Omega$ ，實際電流還受電源與接觸限制。

幫一個遮光提醒器找錯

只在紙上作答，不製造短路，也不更動硬體。

同一組設備、同一位置，室內光十筆範圍 **400 ~ 440**；遮光十筆範圍 **1000 ~ 1040**。以下是為練習設計的資料。

1. 用兩段相近邊界的中點算分界。raw 等於分界時，依本課規則判哪一類？
2. 新 raw=430、800、4095，各會印什麼標籤？哪筆落在兩段基準之間？哪筆先不判？
3. 同學說「raw=800 就是 0.8 V，而且一定遮光」。這句話哪兩個地方不能成立？
4. 供電 3.3 V，上方 1 k Ω 、下方 10 k Ω 。中間相對 GND 多少 V？電表選什麼檔、紅黑筆各碰哪裡？
5. 同學將紅筆插 10A，打算照量電壓的方法跨接 3V3 與 GND。通電前你要阻止哪件事？

題目	我的答案或計算
1	_____
2	_____
3	_____
4	_____
5	_____

下一頁就是解答。先獨立寫，再一起對照理由；這是課堂形成性練習，不新增成績權重。

看數字，也說出判斷的限制

對照上一頁的同一組假資料，不代替自己的實測結果。

1. 分界 720。 $(440 + 1000) \div 2 = 720$ 。遮光在較高一側；本程式低側包含等號，因此 raw=720 判室內光，但它落在兩段基準之間，quality 為 suspect。

2. 分類與原因：

raw	標籤	紀錄該怎麼看
430	室內光	在室內基準內；relative_only
800	遮光	超過 720，但在基準之間；suspect
4095	未判定	端點；suspect，不直接宣稱斷線

3. 兩個問題：raw 是 ADC 整數，不能移動小數點當成精確伏特；800 雖依規則被標成遮光，卻不在任何已觀察基準內，不能保證真實條件。

4. 中間理想 3.0 V。 $3.3 \times 10 \div (1 + 10) = 3.0$ 。黑筆插 COM 並碰 GND 延長端 e3；紅筆插 VΩmA 並碰中間延長端 e23；選直流 20 V。接線前 USB 拔除，核對後才供電量測。

5. 不可用電流插孔 / 檔位跨接電源。保持斷電，將紅筆插回 VΩmA、選直流 20 V，再依正確電壓接法核對。不是只改旋鈕、忘了插孔。

回到作品：哪一筆數據最能讓你看出「程式有標籤」不等於「已確認環境」？說明 raw=800 的原因，比只背答案更重要。

按一下，才記下一筆光線

Week 2 的按鈕，加上本週的 KY-018。不增加新種類的零件。

前面是持續讀光線；現在由你決定何時記錄。可用在比較「紙盒打開 / 蓋上」的光線，不宣稱能自動辨認任何盒子。

用到的舊能力	這次拿來做什麼
Week 2：放開再按、長按只算一次	一次按壓只啟動一批取樣
Week 3：光敏 S 接 ADC	按下時讀取 raw
Serial 文字紀錄	保存批次、筆數、raw 與開機時間

先預測：手一直壓著按鈕，會印一筆，還是一直印？做完再回來回答。

本例與後面的改造題是可選擇的課內延伸，不增加作業分數。先完成安全接線與單模組；課時不足時取代重複操作，不把所有延伸都塞進兩小時。

GPIO5 這次只讀按鈕

本週前面 GPIO5 曾經輸出電壓；不能直接沿用那支程式接按鈕。

1. 拔 USB、拆掉全部外接線；收起分壓電阻與 GPIO5 電壓量測線。
2. 開啟 [完整取樣程式](#)。依本組板卡確認設定後，先只接 USB 上傳。
3. 關閉 Monitor、拔 USB，才依表重接。不要接 5Vin。

設定	圖示板型已確認後的接法
<code>PIN_LIGHT = 4</code>	KY 的 S → a15；c15 → GPIO4
<code>PIN_BUTTON = 5</code>	GPIO5 → 按鈕 A 端；按鈕 B 端 → e3
<code>PROFILE_CONFIRMED = true</code>	只在板型、腳序與 3.3 V 訊號已確認後設定
電源保持分列	板 GND → a3，KY — → b3；板 3V3 → a6，KY 電源 → b6

按鈕沿用 Week 2 斷電通斷測試找出的 A、B 兩組：**放開不通、按下才通**，不是任挑兩腳。可用母端接已辨認的腳並固定絕緣，不讓裸露金屬互碰。

原檔腳位是 -1、確認值 false。圖示 4 / 5 僅供已確認的 YD-ESP32-S3 板使用。GPIO5 設成 INPUT_PULLUP，不是把 3V3 電源直接接到按鈕。

打開、蓋上，各按一次

保持感測器位置不動，只有遮光條件改變。

接 USB，Monitor 選 115200。先放開按鈕至少 0.1 秒，再按住約半秒，確保程式看得到。

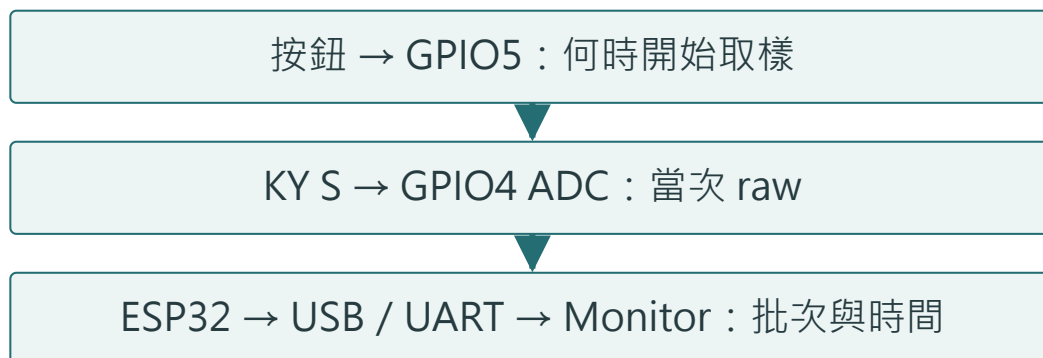
操作	預期觀察；請另記實際結果
室內光，按一次	batch=1、index=1，一筆 raw
按住兩秒	不應新增第二筆
放開，再遮光按一次	batch=2、index=1；raw 大小方向以實物為準
開機時壓住按鈕	先不記錄；放開穩定後重新按才記錄

下面只是格式示例，不是實測：

```
event=sample batch=1 index=1 raw=420 endpoint=0 uptime_ms=3040
event=sample batch=2 index=1 raw=1020 endpoint=0 uptime_ms=6040
```

endpoint=1 表示 raw 是 0 或 4095，需要另查原因；不直接寫成「線斷了」。紀錄留在 Monitor，不會自動存進檔案或 Flash，請匯出到課堂筆記。

按鈕決定何時，感測器決定數字



按鈕和光敏沒有串在同一條線。程式收到一次有效按壓，才向 ADC 要讀數。

按鈕按下時的電流路徑：

3.3 V → 晶片內部上拉電阻 → GPIO5 節點 → 按鈕 → GND

光敏供電回路：

3V3 → KY 的分壓電路 → GND → 板上電源

GPIO4 讀 KY 的中間電壓，不是用它供電。兩個模組共用 GND 作為參考，但 GPIO4 和 GPIO5 不能接在同一列。

程式以 40 ms 等待讀值穩定，再接受按壓；這是本例的選擇，不保證捕捉比它更短的按壓。

按一次，自動取三筆

不改接線，只改同一支程式。完成後再比較資料。

要求：每次按下，立即取第一筆；後續每隔至少 200 ms 各取一筆，共三筆，同一批的 index 依序是 1、2、3。

1. 找到 `SAMPLES_PER_PRESS`，把一筆改成三筆；不要修改 40 ms 按鈕穩定時間。
2. 先預測長按兩秒會出現幾筆，再上傳驗證。
3. 室內光做一批，放開後遮光再做一批。
4. 在一批還沒完成時，再放開並按一次，觀察是否出現 `batch_busy`。
5. 比較每批三筆 `raw`。數字不同代表什麼？能只憑三筆就聲稱測到精確照度嗎？

保存兩批紀錄與一段忙碌時的紀錄。下一頁是參考解答。

批次不變，筆數改成三

```
const int SAMPLES_PER_PRESS = 3;
const uint32_t SAMPLE_GAP_MS = 200;
```

以上取代完整程式中的同名設定，不是把兩行單獨當成新程式。其餘設定保持不變。

測試	應有結果
按住兩秒	一批三筆，不是每 200 ms 永遠取樣
放開後再按	新批次的 index 重新從 1 開始
忙碌時的新按壓	event=ignored reason=batch_busy；不另排隊
後兩筆時間	相鄰取樣至少隔 200 ms，不保證精準等於 200

示例時間 3040、3240、3440 ms 表示同一批跨越約 0.4 秒，**不是三筆同時測量**。實際程式忙碌時，間隔可能更長。

raw 有變化，可能是光線、手的位置或讀取波動。此例未校準照度；三筆都相同也不能證明結果準確。完成後可保留三筆版本，供下週討論「何時量到」的差異。

保存一次完整過程，不重做三份

照片、數值、程式與解釋放在同一份紀錄。

保存什麼	對應本冊段落
板型、GPIO、IDE 版本、組別、電表檔位	器材及操作設定
電源 / GPIO 接線照、3V3 一筆、LOW / HIGH 各三筆、Reset 文字	電源及 GPIO 量測
KY 絲印、三組斷電阻值、接線照、S 電壓三筆	光敏模組辨認及電壓
同輪室內 / 遮光各十筆 raw、條件與電壓	原始資料及範圍
四個範圍值、分界方向、各五筆新驗證	分類規則及限制
1 kΩ / 10 kΩ 實測、A / B 預測與實測、兩張接線照	固定電阻分壓
本週三支程式、討論及練習答案、尚未完成項目	可重現的紀錄

整理順序：條件與接法 → 原始資料 → 比較 → 原因與限制。已有資料直接引用，不為換標題再拍照或重收。未完成寫「未量測 / 未確認」，不填理論值。

最後一起斷電收納

1. 移開表筆、關閉 Monitor、拔 USB 與其他電源，確認 PWR 熄滅。
2. 電表 OFF。先拆 b6 → a20 的供電延長線，再拆分壓電阻與其餘線。
3. 從板子拆下 3V3、GND 線。電阻按阻值收好，模組和線材分開放。

下週再把感測結果接上其他輸出與狀態判斷；本週不把一次分類成功當成可長期自動控制的系統。

停在哪一步，就查那一步

改線前都先斷電。不一次換線、換程式、換設定。

現象	先做什麼
電阻檔顯示 1 / OL	模組保持斷電；檢查接觸、量程與表型說明，不記成 $1\ \Omega$
電壓負號	移開表筆、核對紅黑測點；不反接模組供電
電源與 GND 持續蜂鳴	不通電；依五孔組、導線及元件逐一查路徑
GPIO 文字切換，電壓不變	斷電查 GPIO5 → a12、黑筆 GND，以及已上傳的程式
Monitor 沒文字或亂碼	查 COM 接頭、Port、115200；確認按的是 Upload 而非只有 Verify
gpio_profile_missing	腳位仍是 -1。課前未確認時不猜；設定與圖示接法必須相符
量 S 卻總是電源電壓	紅筆是否量 e6 而非 e15？先移開表筆，改線要斷電
raw 固定 0 / 4095	端點可能飽和或接法有誤；先量 S 對 GND，再斷電查線，不直接判斷斷線
calibration_missing_or_overlap	檢查四個範圍值、端點與重疊；先保留紀錄，不刪不利數字
遮光方向和舊資料不同	分開保存兩輪，查光線、接法、模組及接觸；不混合基準
異常發熱、異味、重啟	立即移除供電，停止使用並保存現象；不反覆通電嘗試

電表量電壓時須供電； Ω / 通斷時不可供電。即使表筆插在同一個 $V\Omega mA$ 孔，旋鈕不同，內部量測方式也不同。

V、A、Ω，不要混在一起

本週用電表量 V 和 Ω；A 只在紙上計算。

問什麼	名稱與單位	例子
兩點電位差多少	電壓 V，伏特	3.3 V
電荷流動多少	電流 A，安培	0.01 A = 10 mA
對電流阻礙多少	電阻 Ω，歐姆	330 Ω；1 kΩ = 1000 Ω

單一電阻的紙上例子：330 Ω 兩端相差 3.3 V，則 $I = 3.3 \div 330 = 0.01 \text{ A}$ ，不是 0.1 A。若 $R = 1 \Omega$ ，理想計算才是 3.3 A；沒外接電阻不能自行當成 1 Ω。

A830L 常見 Ω 量程判讀示例

檔位	示意畫面	解讀
200 Ω	61.0	約 61.0 Ω
20k Ω	1.00	約 1.00 kΩ = 1000 Ω
200k Ω	10.0	約 10.0 kΩ = 10000 Ω
任一 Ω 檔	最左側 1，或其他表的 OL	可能超量程或未接通，不是一般的 1 Ω

實際顯示形式依表型而異，紀錄「61」卻不寫檔位，別人就無法確定單位。電阻檔不是把電壓讀值換成另一個單位。

有聲音就短路嗎？同一五孔組相通，本來就會叫；供電與 GND 原本不該直接相通，卻有低阻路徑，才是需要停止供電排查的情況。

上傳成功，不等於感測成功

三個檢查，回答三個不同問題。

看到什麼	代表什麼	還沒證明什麼
Verify 沒錯誤	電腦能編譯程式	實際接線、電壓與模組功能
Upload 完成	程式已寫入板子	光線分類必定正確
Monitor 與量測符合測試條件	這輪觀察有對應證據	所有器材、環境與時間都可靠

Flash 與 RAM 不要把百分比相加

```
Sketch uses 286973 bytes (21%) ... Maximum is 1310720
Global variables use 21936 bytes (6%) ... Maximum is 327680
```

這是舊編譯摘要的判讀案例，與本次設定或結果不一定相同。

Flash 存程式，斷電仍保留。第一行是在看目前分割設定給程式的空間，不是整顆 16 MB 都拿來當分母。

RAM 是執行時暫存。第二行是編譯時計入的靜態配置，不是之後執行時永遠只用 6%，也不是板上 8 MB PSRAM 的用量。

PSRAM 是額外的 RAM。不是 Flash，也不能由上面兩行就判斷它用了多少。21% 和 6% 分母不同，不能加成整片板用了 27%。

操作依據，和還沒做的測試

查核日期：2026-09-16。以下是來源，不必課堂逐項朗讀。

- [Espressif Arduino-ESP32 ADC](#)：原始值、解析度、衰減與量測範圍。
- [Espressif GPIO](#)：輸入、輸出及 digitalWrite。
- [Espressif ESP32-S3 datasheet](#)：GPIO 與晶片電氣規格。
- [Fluke 電阻量測](#)：斷電與隔離被測物。
- [Fluke 通斷測試](#)：低阻路徑與蜂鳴判讀。
- [Analog Devices：Voltage Divider](#)：分壓電路。
- [Arduino Memory Guide](#)：儲存與執行時記憶體。

板卡、KY 模組、電表與電阻照片來自本課既有實物；板型與焊盤狀態依課堂硬體紀錄，不把賣場名稱當作實物規格。示意圖是依接線關係重新繪製，並非實物比例圖。

本稿整理操作順序與版面；**沒有新增上電、接線、電表讀值或感測器實機測試**。示例數字已標明用途，應填入自己觀察的地方仍保留空白。

教師課前須確認使用板型、GPIO5 輸出與 GPIO4 ADC 的適用接法。既有 BOARD-T01 的觀察不代表全班每片板子都已通過；本稿不解除公開程式中 -1 的保護設定。

後方附上三支程式全文，便於離線閱讀。實際上傳開完整 .ino 檔，不只複製中間某一頁。

week03_gpio_voltage_cycle

第 1 ~ 33 行。所有分頁合起來才是完整程式。

```
// Fill in only the GPIO value published in the verified course hardware profile.
// Keep -1 when no verified profile is available.
const int PIN_TEST_OUTPUT = -1;
const unsigned long PHASE_DURATION_MS = 10000;

bool outputHigh = false;
unsigned long phaseStartedMs = 0;
unsigned long cycleNumber = 1;

bool profileReady() {
    return PIN_TEST_OUTPUT >= 0;
}

void reportPhase() {
    Serial.printf(
        "cycle=%lu phase=%s expected_voltage=%s measure_now=true\n",
        cycleNumber,
        outputHigh ? "HIGH" : "LOW",
        outputHigh ? "near_3.3V" : "near_0V"
    );
}

void setup() {
    Serial.begin(115200);
    delay(500);

    if (!profileReady()) {
        Serial.println("week=3 status=blocked reason=gpio_profile_missing");
        return;
    }

    pinMode(PIN_TEST_OUTPUT, OUTPUT);
    digitalWrite(PIN_TEST_OUTPUT, LOW);
```

下一頁接續同一支程式，不另開草稿。

week03_gpio_voltage_cycle

第 34 ~ 56 行。所有分頁合起來才是完整程式。

```
outputHigh = false;
phaseStartedMs = millis();

Serial.printf(
    "week=3 pin_test_output=%d startup=LOW phase_duration_ms=%lu\n",
    PIN_TEST_OUTPUT,
    PHASE_DURATION_MS
);
reportPhase();
}

void loop() {
    if (!profileReady()) return;

    unsigned long now = millis();
    if (now - phaseStartedMs < PHASE_DURATION_MS) return;

    phaseStartedMs = now;
    outputHigh = !outputHigh;
    digitalWrite(PIN_TEST_OUTPUT, outputHigh ? HIGH : LOW);
    if (!outputHigh) cycleNumber++;
    reportPhase();
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。

week03_ky018_raw

第 1 ~ 32 行。所有分頁合起來才是完整程式。

```
// Fill in only the ADC GPIO published in the verified course hardware profile.
// Keep -1 when no verified profile is available.
const int PIN_LIGHT = -1;
const unsigned long SAMPLE_INTERVAL_MS = 500;
const char* DEVICE_ID = "CHANGE_ME";

unsigned long lastSampleMs = 0;
unsigned long sampleNumber = 0;

bool profileReady() {
    return PIN_LIGHT >= 0;
}

void setup() {
    Serial.begin(115200);
    delay(500);

    if (!profileReady()) {
        Serial.println("week=3 sensor=ky018 status=blocked reason=gpio_profile_missing");
        return;
    }

    analogReadResolution(12);
    analogSetPinAttenuation(PIN_LIGHT, ADC_11db);
    Serial.printf(
        "week=3 sensor=ky018 status=ready device=%s pin_light=%d "
        "adc_bits=12 interval_ms=%lu\n",
        DEVICE_ID,
        PIN_LIGHT,
        SAMPLE_INTERVAL_MS
    );
}
```

下一頁接續同一支程式，不另開草稿。

week03_ky018_raw

第 33 ~ 50 行。所有分頁合起來才是完整程式。

```
void loop() {  
    if (!profileReady()) return;  
  
    unsigned long now = millis();  
    if (now - lastSampleMs < SAMPLE_INTERVAL_MS) return;  
    lastSampleMs = now;  
    sampleNumber++;  
  
    int lightRaw = analogRead(PIN_LIGHT);  
    Serial.printf(  
        "device=%s sample=%lu uptime_ms=%lu light_raw=%d\n",  
        DEVICE_ID,  
        sampleNumber,  
        now,  
        lightRaw  
    );  
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。

week03_light_classifier

第 1 ~ 31 行。所有分頁合起來才是完整程式。

```
// Relative light classification: calibration ranges come from the student's log.
// No pin is approved by compiling this program. Keep -1 until a profile is published.
#include <Arduino.h>

const int PIN_LIGHT = -1;
const char *DEVICE_ID = "CHANGE_ME";
const int INDOOR_MIN = -1;
const int INDOOR_MAX = -1;
const int SHADE_MIN = -1;
const int SHADE_MAX = -1;
const unsigned long SAMPLE_INTERVAL_MS = 500;
bool ready = false;
bool shadeHigher = true;
int threshold = -1;
unsigned long lastSampleMs = 0;
unsigned long sample = 0;

bool configureCalibration() {
    // End-point values are excluded by this classroom policy, not proof of a fault.
    if (INDOOR_MIN <= 0 || SHADE_MIN <= 0 || INDOOR_MAX >= 4095 ||
        SHADE_MAX >= 4095 || INDOOR_MIN > INDOOR_MAX || SHADE_MIN > SHADE_MAX)
        return false;
    if (INDOOR_MAX < SHADE_MIN) {
        shadeHigher = true;
        threshold = (INDOOR_MAX + SHADE_MIN) / 2;
    } else if (SHADE_MAX < INDOOR_MIN) {
        shadeHigher = false;
        threshold = (SHADE_MAX + INDOOR_MIN) / 2;
    } else return false; // Overlap or touching ranges cannot be separated here.
    return true;
}
```

下一頁接續同一支程式，不另開草稿。

week03_light_classifier

第 32 ~ 63 行。所有分頁合起來才是完整程式。

```
const char *classifyLight(int raw) {
    if (!ready || raw <= 0 || raw >= 4095) return "UNDECIDED";
    const bool lowSide = raw <= threshold;
    return (shadeHigher ? !lowSide : lowSide) ? "SHADE" : "INDOOR";
}

const char *chineseLabel(const char *state) {
    if (strcmp(state, "SHADE") == 0) return "遮光";
    if (strcmp(state, "INDOOR") == 0) return "室內光";
    return "未判定";
}

const char *qualityReason(int raw) {
    if (raw <= 0 || raw >= 4095) return "adc_endpoint";
    const int observedMin = min(INDOOR_MIN, SHADE_MIN);
    const int observedMax = max(INDOOR_MAX, SHADE_MAX);
    if (raw < observedMin || raw > observedMax) return "outside_observed_span";
    const bool inIndoor = raw >= INDOOR_MIN && raw <= INDOOR_MAX;
    const bool inShade = raw >= SHADE_MIN && raw <= SHADE_MAX;
    if (!inIndoor && !inShade) return "between_baselines";
    return "within_baseline_range";
}

void setup() {
    Serial.begin(115200);
    delay(500); // Only startup; sampling below does not block for half a second.
    if (PIN_LIGHT < 0) {
        Serial.println("week=3 status=blocked reason=gpio_profile_missing");
        return;
    }
    if (!configureCalibration()) {
```

下一頁接續同一支程式，不另開草稿。

week03_light_classifier

第 64 ~ 88 行。所有分頁合起來才是完整程式。

```
    Serial.println("week=3 status=blocked reason=calibration_missing_or_overlap");
    return;
}
analogReadResolution(12);
analogSetPinAttenuation(PIN_LIGHT, ADC_11db);
ready = true;
lastSampleMs = millis();
Serial.printf("week=3 status=ready device=%s threshold=%d shade_higher=%s
interval_ms=%lu scope=local_only\n",
              DEVICE_ID, threshold, shadeHigher ? "true" : "false",
              SAMPLE_INTERVAL_MS);
}

void loop() {
    if (!ready) return;
    const unsigned long now = millis();
    if (now - lastSampleMs < SAMPLE_INTERVAL_MS) return;
    lastSampleMs = now; // Late iterations take one current sample, not fake catch-up
    samples.
    const int raw = analogRead(PIN_LIGHT);
    const char *state = classifyLight(raw);
    const char *reason = qualityReason(raw);
    const bool plausible = raw > 0 && raw < 4095;
    Serial.printf("device=%s sensor=ky018 source=hardware sample=%lu uptime_ms=%lu
light_raw=%d threshold=%d class=%s label=%s valid=%s quality=%s reason=%s\n",
                  DEVICE_ID, ++sample, now, raw, threshold, state, chineseLabel(state),
                  plausible ? "true" : "false",
                  strcmp(reason, "within_baseline_range") == 0 ? "relative_only" :
"suspect", reason);
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。

button_light_capture

第 1 ~ 34 行。所有分頁合起來才是完整程式。

```
#include <Arduino.h>

// Fill only after checking the actual board and the powered-off wiring.
const bool PROFILE_CONFIRMED = false;
const int PIN_LIGHT = -1;
const int PIN_BUTTON = -1;
const int SAMPLES_PER_PRESS = 1;
const uint32_t SAMPLE_GAP_MS = 200;
const uint32_t DEBOUNCE_MS = 40;

bool ready = false, armed = false, collecting = false;
int lastButton = HIGH, sampleIndex = 0;
uint32_t changedAt = 0, lastSampleAt = 0, batch = 0;

bool pressed(uint32_t now) {
    const int level = digitalRead(PIN_BUTTON);
    if (level != lastButton) {
        lastButton = level;
        changedAt = now;
    }
    if (uint32_t(now - changedAt) < DEBOUNCE_MS) return false;
    if (level == HIGH) { armed = true; return false; }
    if (!armed) return false;
    armed = false;
    return true;
}

void capture(uint32_t now) {
    const int raw = analogRead(PIN_LIGHT);
    ++sampleIndex;
    Serial.printf("event=sample batch=%lu index=%d raw=%d "
                  "endpoint=%d uptime_ms=%lu\n",
                  (unsigned long)batch, sampleIndex, raw,
                  raw == 0 || raw == 4095, (unsigned long)now);
}
```

下一頁接續同一支程式，不另開草稿。

button_light_capture

第 35 ~ 58 行。所有分頁合起來才是完整程式。

```
    lastSampleAt = now;
    if (sampleIndex >= SAMPLES_PER_PRESS) collecting = false;
}

void setup() {
    Serial.begin(115200);
    if (!PROFILE_CONFIRMED || PIN_LIGHT < 0 || PIN_BUTTON < 0 ||
        PIN_LIGHT == PIN_BUTTON || SAMPLES_PER_PRESS < 1 ||
        SAMPLES_PER_PRESS > 10) {
        Serial.println("event=blocked reason=check_configuration");
        return;
    }
    pinMode(PIN_BUTTON, INPUT_PULLUP);
    analogReadResolution(12);
    analogSetPinAttenuation(PIN_LIGHT, ADC_11db);
    changedAt = millis();
    ready = true;
    Serial.println("event=ready release_button_first");
}

void loop() {
    if (!ready) return;
    const uint32_t now = millis();
    if (pressed(now)) {
```

下一頁接續同一支程式，不另開草稿。

button_light_capture

第 59 ~ 70 行。所有分頁合起來才是完整程式。

```
    if (collecting) {
        Serial.println("event=ignored reason=batch_busy");
    } else {
        ++batch;
        sampleIndex = 0;
        collecting = true;
        capture(now);
    }
}
if (collecting && uint32_t(now - lastSampleAt) >= SAMPLE_GAP_MS)
    capture(now);
}
```

這支程式到此結束。原始檔保留未確認腳位的保護值；不要把編譯成功當成實機通過。